# UAC-AD: Unsupervised Adversarial Contrastive Learning for Anomaly Detection on Multi-Modal Data in Microservice Systems

Hongyi Liu, Xiaosong Huang, Mengxi Jia, Tong Jia, Jing Han, Zhonghai Wu, and Ying Li, *Member, IEEE*

***Abstract*—To ensure the stability and reliability of microservice systems, timely and accurate anomaly detection is of utmost importance. Recently, considering the lack of labels in real-world scenarios and the collaborative and complementary relationships of multi-modal data in reflecting system anomalies, unsupervised multi-modal anomaly methods have been proposed. However, existing methods face challenges in effectively distinguishing normal hard samples (they are normal but hard to classify correctly) from anomalies. This is mainly caused by two aspects. First, the hard sample patterns are complex. Second, the convergence speed is inconsistent between hard and simple samples. To overcome these issues, we propose an unsupervised adversarial contrastive multi-modal anomaly detection method (UAC-AD). We utilize contrastive learning to help learn the complex patterns of hard samples and enlarge the distance between hard and anomaly samples. Meanwhile, the adversarial framework automatically identifies hard samples and fine-grained adjusts the training weights to each modality part of these hard samples. In this case, The hard sample problems of two aspects can be alleviated. We extensively evaluate UAC-AD on two open-source simulated datasets and a real industrial dataset from a large communication company. Extensive experimental results demonstrate the effectiveness of our approach in anomaly detection. We also release the code and dataset for replication and future research.**

***Index Terms*—Adversarial learning, anomaly detection, contrastive learning, microservice systems, multi-modal data, software reliability, unsupervised learning.**

## I. Introduction

RECENTLY, with more and more online applications migrating to cloud platforms, microservice architecture has received great attention. They are widely adopted due to their capability to allow development, deployment, update, and scale for each service. A microservice system is a large system with many instances (e.g., virtual machines or containers). However, with the scale and complexity of microservice systems expanding rapidly, failure is inevitable. When an instance fails, it may degrade the performance of the entire system, impact user experience, and result in substantial financial losses. Therefore, it is crucial to proactively detect anomalies and mitigate failures to guarantee the reliability of the microservice systems. In real-world microservice systems, many types of monitoring data, including metrics, logs, alerts, and traces, play an essential role in software reliability engineering. Especially for log and metric data, operators continuously collect them for prompt anomaly detection. Metric data includes system-level metrics (such as CPU utilization, memory usage, and network bandwidth) and user-perceived metrics (like average response time, error rate, etc.). Logs are semi-structure messages printed, which typically capture operational information about hardware and software, including state changes, debug outputs, and system alerts.

As manually identifying anomalies is impractical and prone to errors, significant efforts are invested in automated anomaly detection. Some methods rely on metrics to detect anomalies by probability density estimation, outlier detection, or autoregressive bias (more details are introduced in subsection II-A). Other methods rely on logging to detect anomalies through keyword matching or learning-based methods (more details are introduced in subsection II-B). However, in the context of microservice systems, relying on only one single modal data is insufficient. It is unable to depict the status of systems preciously to judge whether anomalies occur [1], [2]. Existing unsupervised multi-modal anomaly detection methods mainly rely on reconstruction-based approaches [3], assuming that normal data is easier to reconstruct than anomalous data. These methods learn the distribution of normal data by fitting unlabeled data and identifying anomalies based on reconstruction errors.

However, these existing reconstruction-based multi-modal anomaly detection methods struggle to distinguish between hard and anomalous samples in the decision space, leading to limited anomaly detection performance. In our work, hard samples refer to normal hard samples, which are a part of normal samples, but are difficult for the model to classify correctly and can be easily classified as abnormal samples [4], [5]. The underlying reasons for this limitation are twofold: (1) *Complex Combination:* In microservice systems, the combination of normal samples is complex. (one log pattern can be combined with multiple metric

Manuscript received 25 October 2023; revised 10 May 2024; accepted 28 May 2024. Date of publication 7 June 2024; date of current version 30 December 2024. This work was supported by the PKU-ZTE Cooperation Research Rroject. *(Hongyi Liu, Xiaosong Huang, and Mengxi Jia contributed equally to this work.) (Corresponding author: Ying Li.)*

Hongyi Liu, Mengxi Jia, Zhonghai Wu, and Ying Li are with the School of Software & Microelectronics, Peking University, Beijing 100871, China (e-mail: hongyiliu@pku.edu.cn; mxjia@pku.edu.cn; wuzh@pku.edu.cn; li.ying@pku.edu.cn).

Xiaosong Huang is with the School of Computer Science, Peking University, Beijing 100871, China (e-mail: hxs@stu.pku.edu.cn).

Tong Jia is with Institute for Artificial Intelligence, Peking University, Beijing 100871, China (e-mail: jia.tong@pku.edu.cn).

Jing Han is with the Department of Algorithm, ZTE, Shenzhen 518083, China (e-mail: han.jing28@zte.com.cn).

Digital Object Identifier 10.1109/TSC.2024.3411481

1939-1374 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

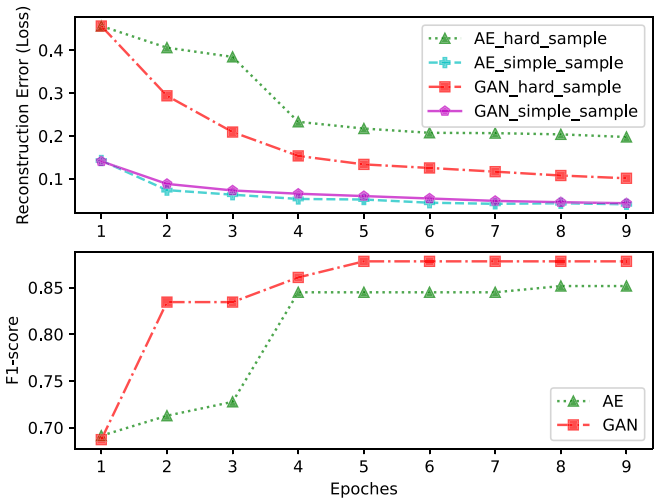



Fig. 1. The Motivation of UAC-AD. The figure illustrates the fitting process of Autoencoder (AE, i.e., Method5 in Table III) and GAN (i.e., Method6 in Table III) for hard and simple samples in the training set of Dataset A (introduced in section IV-A1). The hard samples are marked according to the false positives identified by the AE-based anomaly detection model that completed the first epoch of reconstruction training, while the rest samples are considered as simple samples. The top figure demonstrates the issue of inconsistent convergence speed between multi-modal hard and simple samples: both AE and GAN converge in two epochs for simple samples, whereas hard samples require more epochs to converge. The bottom figure presents the performance of AE and GAN for anomaly detection. These two figures demonstrate that adversarial learning could adjust the convergence speed between hard and simple samples and lead to a improved anomaly detection performance.

patterns to represent normal system states, and similarly, one metric pattern can also be combined with multiple log patterns to represent normal system states). In consideration that hard samples are part of normal samples, the combination of hard samples is also complex. Existing reconstruction-based methods fail to capture diverse and complex combinations of hard samples, which may identify these hard samples as anomalies. (2) *Inconsistent Convergence Speed:* As shown in Fig. 1, the convergence speed of simple and hard samples in multi-modal data are inconsistent, leading to models overfitting simple samples and underfitting hard samples, thus limiting the model's ability to model normal samples and consequently restricting the effectiveness of anomaly detection. For reconstruction-based anomaly detection methods, multi-modal hard samples may exhibit large reconstruction errors in both modalities or in one single modality. For the latter case, fine-grained (modality-level rather than sample-level) training optimization adjustments are required to balance the fitting degrees of both modalities, achieving unified modeling of normal samples and obtaining better anomaly detection performance.

To address the two challenges of hard samples mentioned above, we propose a novel unsupervised adversarial contrastive learning-based method for general log-metric multi-modal data anomaly detection (UAC-AD). Specifically, we propose an adversarial learning framework with contrastive learning. We utilize contrastive learning is to learn the complex combinations of normal samples by enlarging the distance between normal and anomaly samples so that the model can better distinguish between hard and abnormal samples, reduce false positives, and improve the performance of anomaly detection. Some studies [6] have confirmed that time-aligned multi-modal data reflects the operational state of the system, while unaligned data reflects inconsistent system states. Therefore, we consider the time-aligned combination of logs and metrics as positive (i.e., normal) samples and treat the unaligned combination as negative (i.e., abnormal) samples. Then we increase the distance between positive and negative samples, alleviating the first challenge of hard samples.

Moreover, in our adversarial training phase, the discriminator can judge the reconstruction performance of the generator and increase the weight of reconstruction learning for the parts with poor reconstruction performance (i.e., hard samples for the reconstruction-based anomaly detection methods). We furthermore design separate discriminators against each modality part of the multi-modal data to achieve a modality-level training optimization adjustment for hard samples.

Our contributions are as follows:

- We clarify the problem of inconsistent convergence speed of hard samples for multi-modal anomaly detection in microservice systems, and we design a fine-grained adversarial learning framework to adjust the convergence speed of each modality part of multi-modal hard samples. Extensive ablation studies verify the effectiveness of the proposed adversarial learning strategy.
- Within the adversarial learning framework, we introduce contrastive learning to enhance the model's understanding of complex combinations of the multi-modal hard samples. This widens the gap between hard and anomalous samples in the decision space, thereby achieving better anomaly detection performance. Moreover, we release our code and related data sets for better replication and future research [7].

## II. Related Work

Recently, tremendous efforts have been devoted to anomaly detection to ensure the reliability of large-scale systems. The anomaly detection methods are usually based on logs, metrics, or both. In this section, we first review the anomaly detection works, including metric-based, logging-based, and multi-modal-based methods, which are closely related to our work. We then introduce the key techniques we used in this paper, including adversarial learning and contrastive learning.

### A. Metric-Based Methods

Metric data is a typical time series data collected from the monitors to monitor the running state of the instances at the application or system level. According to the criterion for anomaly determination, the paradigms of metric-based anomaly detection can be categorized into three types. *Density estimation-based methods* [8], [9], [10], [11] assumed that the normal data conforms to a specific probability distribution and identified anomalies according to the probability density of the data points or the likelihood of the data points appearing. Zong et al. [10] and Yairi et al. [11] introduced the Gaussian mixture models into their framework, facilitating the estimation of representation

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

densities. *Clustering-based methods* [12], [13], [14] assumed that the outlier data is the anomaly and identified the anomalies according to the distance from the data point to the cluster center. Tax et al. [12] and Ruff et al. [13] constructed a cluster from the representation of the normal data with nonlinear transformation. *Reconstruction-based methods* [15], [16], [17], [18], [19], [20] assumed that the abnormal data is difficult to reconstruct, and determined the anomalies according to the reconstruction error. Park et al. [15], and Ya et al. [16] used the LSTM or GRU to capture the temporal dependency and the Variational AutoEncoder (VAE) to reconstruct. Hang et al. [17], and Jun et al. [18], both of which incorporated Graph Attention Networks (GAT) to capture spatial correlations among dimensions in multivariate time series. Estimation-based methods presuppose specific data distributions, clustering-based methods rely on hyperparameter configurations, and reconstruction-based methods are vulnerable to hard samples within training data. Consequently, deploying these approaches in practical production environments presents formidable challenges.

### B. Log-Based Methods

Log data is semi-structured text collected by instances at the application or system level. Logs are widely adopted in practice for anomaly detection. Traditional log-based anomaly detection methods are usually designed to identify keywords in logs like "error" or "fail" or count the number of logs that appear during a period of time. However, negative keywords or the number of logs within a period of time could not accurately imply instance failures. Thus, advanced approaches are proposed, which follow a similar workflow: log parsing, feature extraction, and anomaly detection. These works mine the log patterns (e.g., sequential feature, semantic feature) of normal executions and judge whether an anomaly occurs when the current execution deviates from the learned normal execution. For example, Xu et al. [21] constructed normal and abnormal space of log event count matrix using Principal Component Analysis (PCA) to detect anomalies. Lin et al. [22] and He et al. [23] designed clustering-based methods to identify problems of online service systems. Du et al. [24] predicted the logs that may appear after a sliding window utilizing the LSTM model. Zhang et al. [25] took the entire sequence into account and trained an attention-based Bi-LSTM model on the log sequence for supervised learning. Wei et al. [26] employed feature engineering to extract log feature vectors. They utilized a GRU-based Autoencoder to learn reconstruction and identified anomalies based on reconstruction errors. Yuanyuan et al. [27] extracted semantic features of logs using Transformers. They employed LSTM and CNN to capture both global and local correlations within log sequences, enhancing the performance of supervised anomaly detection. However, existing methods for log-based anomaly detection are constrained by their reliance on singular data sources, making it challenging to meet the real-world demand for accuracy.

### C. Multi-Modal Based Methods

Although single-modal-based anomaly detection methods are confirmed effective, [28] demonstrated that it may neglect some anomalies when only relying on single data. The Faults can cause unexpected behaviors involving either logs, metrics, or both of them. So, to achieve better performance, the researchers attempt to analyze more than one data source comprehensively to reveal the actual anomalies. Some research [1], [29], [30], [31], [32] proposed integrating tracing data with logs or metrics to group logs or metrics from the same execution context, aiming to better learn data patterns in microservices systems and assist in pinpointing faulty nodes. However, acquiring tracing data requires pre-instrumentation, making it challenging to apply to already deployed microservices. Moreover, the collection of tracing data imposes additional overhead on running services, potentially constraining service response times. Lee et al. [28] proposed a log-metric modal anomaly detector via a semi-supervised learning method to effectively detect anomalies. In consideration of scarce label data in practice, Zhao et al. [3] and Chen et al. [33] designed unsupervised anomaly detection methods based on log and metric data with a concatenation feature fusion strategy and reconstruction-based criterion. Zhang et al. [34] conducted autoregressive anomaly detection separately based on log and metric modalities and fused the results according to anomaly density. However, existing unsupervised methods struggle to adapt to the intricate data disparities between multiple modalities and are susceptible to the influence of hard samples, restricting the accuracy of anomaly detection.

### D. Adversarial Learning

Adversarial Learning, exemplified by the Vanilla Generative Adversarial Network (GAN) proposed by Goodfellow et al. [35], is a fundamental technique in machine learning. This approach involves a generator and a discriminator. The generator reconstructs input data and attempts to deceive the discriminator, while the discriminator discerns whether the input data originates from real or generated sources. Let $G$ denote the generator network, and $D$ denote the discriminator network. The optimization objective of GAN is:

$$\min_G \max_D V(G,D) = \mathbb{E}_{x\sim p(x)}[log(D(x))] + \mathbb{E}_{x\sim p(x)}[log(1-D(G(x)))]. \quad (1)$$

Numerous studies have explored the application of GANs in anomaly detection, primarily focusing on unimodal data types such as images [36], [37] and time series [19], [20]. In contrast to these single-modal approaches, our proposed method delves into the realm of multi-modal data, investigating how GANs can be harnessed to enhance the effectiveness of anomaly detection across diverse data modalities.

### E. Contrastive Learning

Contrastive learning is a self-supervised learning method in deep learning that does not require extra annotation information to augment the representation of data. Triplet loss [38] is a typical method to achieve contrastive learning, which reduces the distance between $x$ and $x^+$, and increases the distance between $x$ and $x^-$, as shown in (2). Where $x^+$ is a positive sample that is similar to $x$, $x^-$ is a negative sample that is dissimilar to $x$, $\alpha$ is a hyperparameter used to adjust the difference between

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

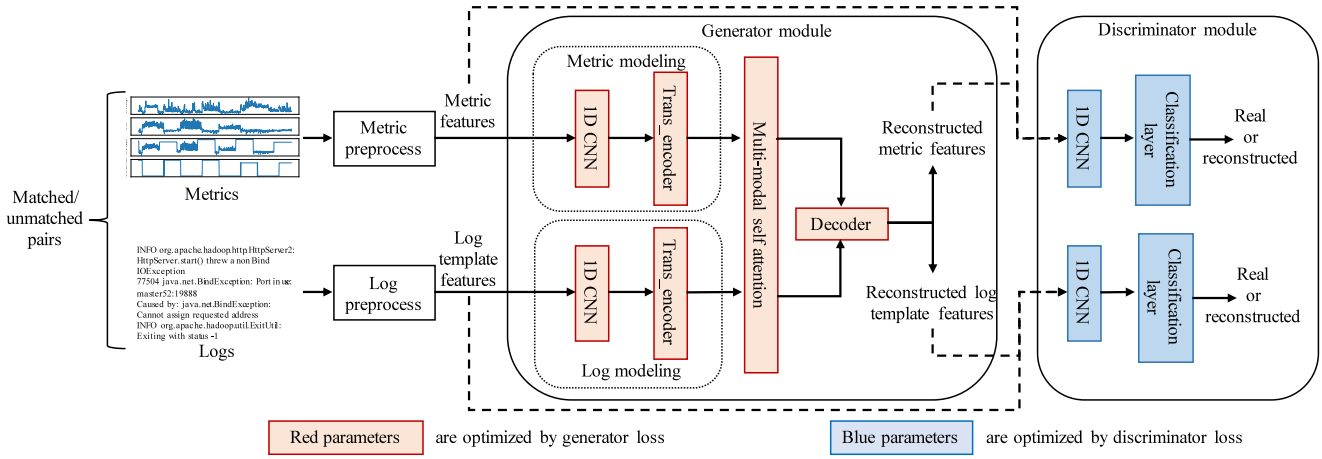



Fig. 2. The UAC-AD pipeline comprises a generator and a discriminator module. The goal is to accurately fit the normal sample distribution and distinguish anomalies utilizing multi-modal data through an adversarial mechanism. The generator, with four components, i.e., log modeling, metric modeling, multi-modal self-attention, and decoder, aims to fit multi-modal normal data through reconstruction. The discriminator, comprising log and metrics discriminator, effectively guides the generator by distinguishing between original and reconstructed data.

two distances.

$$L = \max(0, d(f(x), f(x^+)) - d(f(x), f(x^-)) + \alpha). \quad (2)$$

To our knowledge, there are very few works [6], [39] that attempt to use contrastive learning in multi-modal anomaly detection for microservices. The most relevant work, CL-MMAD [6], employs CNN and LSTM to extract features from metrics and traces separately. It then utilizes contrastive learning to narrow down the distance between these features in the feature space and employs the distance of both features as anomaly scores. This approach uses contrastive loss as the sole training loss, enabling it to detect anomalies related to modal inconsistencies but struggling to adapt to anomalies that exhibit modal consistency but temporal anomalies. In our work, we fuse multi-modal features through time alignment. We treat time blocks with matched logs and metrics as positive samples and those with unmatched logs and metrics (generated in different time blocks) as negative samples. This fusion contributes to the reconstruction of multi-modal input data within time windows, enabling further learning of the temporal correlations in multi-modal data. Consequently, our approach yields improved representations of multi-modal data and better overall performance.

## III. Methodology

### A. Overview

Our approach consists of two key modules, the *Generator* and the *Discriminator*, as illustrated in Fig. 2. The *Generator* serves as an autoencoder and is employed to reconstruct the original data. The *Discriminator* acts as a classifier, distinguishing between original and reconstructed data, thereby effectively guiding the *Generator* to fit the normal sample distribution. The overview of the training and inference phases is illustrated in Fig. 3.

- In the training phase, the *Generator* leverages a combination of reconstruction loss, adversarial loss, and contrastive loss. The reconstruction loss is employed to learn the distribution of normal data by minimizing the error between input and output data. Simultaneously, the adversarial loss is employed to deceive the discriminator, enabling the generator to automatically adjust the convergence speeds for hard samples and simple samples. This further facilitates the fitting of the distribution of normal data. Moreover, the contrastive loss is utilized to aid the generator in learning multi-modal data combinations of log and metric data that correspond to normal operation states in microservices. This is achieved by preventing the learning of abnormal combinations, thereby enhancing the differentiation between normal and abnormal data. This contrastive loss is computed based on model outputs for matched pairs and unmatched pairs. A matched pair, comprising logs and metric data collected at the same time block, represents a unified system state and is considered normal. In contrast, an unmatched pair consists of logs and metrics collected at different time blocks, considered abnormal. The

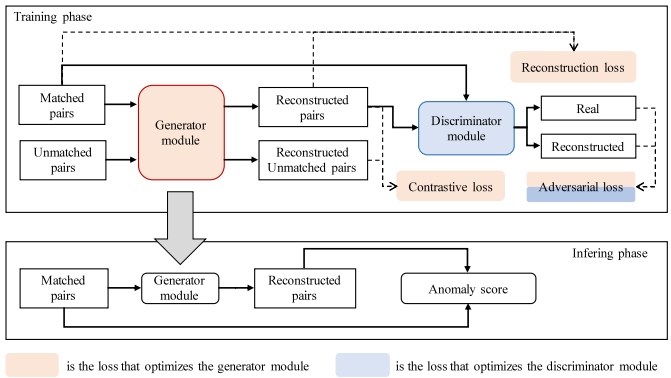



Fig. 3. Illustration of the training and inferring phase of UAC-AD. The training phase incorporates multi-modal data modeling, reconstruction, and discrimination. The generator uses a mix of losses to fit the normal data, with the discriminator differentiating between real and generator-generated samples to refine this fitness. The inferring phase centers on the generator's reconstruction errors. Surpassing a predefined threshold indicates an anomaly.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

*Discriminator* employs adversarial loss to distinguish whether input data originates from real or Generator-generated data. For hard samples that are difficult to reconstruct and have larger reconstruction errors, the discriminator can distinguish them between real or reconstructed well. To deceive the discriminator, the generator is pushed to fit the normal data distribution closely.

• In the inference phase, we primarily focus on the generator's reconstruction errors, which indicates the gap between real data and reconstructed data. When these errors exceed a predefined threshold, we classify it as an anomaly, triggering an alert to notify operations personnel.

The proposed method follows an instance-based approach rather than a system-based approach. Therefore, we apply our model separately to the real-time data generated by each service instance within the system for anomaly detection. If any instance reports an anomaly, we consider the entire system to be anomalous. The advantage of this instance-based approach is that it allows us to pinpoint the specific instance(s) experiencing anomalies while also avoiding issues related to inconsistent timestamps across different services. Many existing methods for anomaly detection in logs and metrics, such as Hades [28], OmniAnomaly [16], and DeepLog [24], also adopt an instance-based approach.

### B. Data Preprocessing

Given a sequence of logs and metrics from a microservice system, the first challenge of fusion multi-modal data is to align the logs and metrics at the same time block to indicate the same system state. The metrics are in the typical time series format with a certain sampling interval, which can be easily handled. However, the logs are semi-structured or unstructured messages (texts) with the corresponding timestamp, which contains variables and impedes log analysis. Therefore, we adopt a widely adopted parser called Drain [40] to extract log events, as Drain has been proven to be effective and efficient in past evaluations. We first utilize the Drain to extract $n$ log templates, and then when a log message arrives, we could match it to the corresponding template. Subsequently, all log messages are converted into log events arranged in chronological order. Then, we align the logs and metrics by setting the sampling interval of metrics as the size of the aligned time block and assigning all log events and metrics to their respective time blocks.

Then, we uniform the data structure of log data and metric data by transforming log data into time series data. For each time block, we assess the occurrence of each log template, marking it as 1 if it appears and 0 otherwise. By doing so, we obtain a $1 \times n$ dimensional template feature for the specific time block, where the $n$ is the number of template types. The motivation behind this stems from the fact that there is a significant difference between the log templates in the normal and abnormal system status. For example, when it comes to abnormal system status, there are log templates containing error handling and fault tolerance-related information, while these log templates rarely appear in the normal system state. Therefore, we focus on the occurrence of each template, capturing the co-occurrence relationships among templates and leveraging them to differentiate different system statuses (normal or abnormal).

Through the above data process, we convert raw logs and metrics to aligned matched time series data, $X = (X_l, X_m) = \{x^1, x^2, \ldots, x^t\}$, where $X \in \mathbb{R}^{t\times(n+n')}$ indicates the multi-modal data, $X_l \in \mathbb{R}^{t\times n}$ indicates the processed log template features, $X_m \in \mathbb{R}^{t\times n'}$ indicates the sampled metrics, $t$ indicates the total number of time blocks, $n$ indicates the total number of log template types, $n'$ indicates the total number of metrics. The goal of anomaly detection in multi-modal data is to construct a model $F(X^i)$ to predict whether $X^i$ contains anomalies or not, where $X^i = \{x^i, x^{i-1}, \ldots, x^{i-w-1}\}$ indicates the multi-modal data of the $i_{th}$ slide window, the $w$ denotes the slide window size.

### C. Generator Module

The structure of the generator module is shown in Fig. 2. It is divided into four parts: log modeling, metric modeling, multi-modal self-attention, and decoder.

*Log Modeling:* We designed a log encoder module to encode the relationships between time blocks further. For $i_{th}$ sliding window, we adopt a Convolutional Neural Network (CNN) to map low-dimensional features to high-dimensional features. Then we apply a Transformer encoder to learn the time-association features between blocks, and then we obtain the log hidden layer representation for the $i_{th}$ sliding window $Z_l^i \in \mathbb{R}^{w\times h}$, as shown in (3). Where $X_l^i$ indicates the $i_{th}$ slide window of the log template feature, $h$ denotes the hidden size of the neural network.

$$Z_l^i = \text{TransEncoder}(\text{CNN}(X_l^i)). \tag{3}$$

*Metric Modeling:* Similar to Log Modeling, we use CNN and Transformer encoder to capture the correlation between indicators and generate metric hidden layer representations $Z_m^i \in \mathbb{R}^{w\times h}$, as shown in (4). Where $X_m^i$ indicates the $i_{th}$ slide window of sampled metric.

$$Z_m^i = \text{TransEncoder}(\text{CNN}(X_m^i)). \tag{4}$$

*Multi-modal Self-attention:* In order to learn the correlation between hidden variables in different modalities, we consider concatenating the hidden layer representations of both modalities and applying self-attention fusion, resulting in a multi-modal self-attention (MSA) mechanism. The formulas are as follows:

$$\begin{aligned} Z_f^i &= \text{Cat}(Z_l^i, Z_m^i), \\ Q, K, V &= Z_f^i W_Q, Z_f^i W_K, Z_f^i W_V, \\ Z_f'^i &= \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V, \\ Z_f''^i &= \text{LayerNorm}(Z_f^i + Z_f'^i), \end{aligned} \tag{5}$$

where $Z_l^i$ and $Z_m^i$ are the hidden layer representations of the two modalities. Cat() represents the concatenate function in the last axis. $Z_f^i \in \mathbb{R}^{w\times 2h}$ is the concatenated representation of $Z_l^i$ and $Z_m^i$. $W_Q, W_K, W_V \in \mathbb{R}^{d\times d}$ represent the trainable hyper-parameters, and Q, K, and V are the linear projections of $Z_f^i$. $d$

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

indicates the hidden size of the attention layer (in there, $d$ equals $2h$). $Z_f'^i \in \mathbb{R}^{w\times 2h}$ is the output after attention operation. Finally, we use residual connections [41] and layer normalization [42] to generate the ultimate fused features $Z_f''^i \in \mathbb{R}^{w\times 2h}$.

*Decoder:* To achieve data reconstruction, we map the fused features back to the input space. Specifically, we employ a linear transformation to map $Z_f''^i$ to $R^i \in \mathbb{R}^{k\times(n+n')}$, and split $R^i$ into $R_l^i \in \mathbb{R}^{k\times n}$ and $R_m^i \in \mathbb{R}^{k\times n'}$. $R_l^i$ and $R_m^i$ are the reconstructed values of $X_l^i$ and $X_m^i$, respectively. The formula is shown in (6):

$$\begin{aligned} R^i &= \sigma(W_d Z_f''^i + b_d), \\ R_l^i, R_m^i &= \text{Split}(R^i), \end{aligned} \tag{6}$$

where $R$ denotes the reconstruction of $X$, $\sigma()$ denotes the ReLU activation function, the $W_d \in \mathbb{R}^{2h\times(n+n')}$ and $b_d \in \mathbb{R}^{(n+n')}$ are the learnable parameters, and the Split() indicates the split operation of $R$ in the last axis.

### D. Discriminator Module

In the framework of our discriminator, which serves to differentiate real data from generated (fake) data, we introduce two individual distinct components: the Log Discriminator and the Metrics Discriminator.

*Log Discriminator:* The log discriminator, denoted as $D_l$, consists of a feature embedding layer and a linear classification layer. It employs a CNN to embed log template features $I_l^i$ into a hidden representation denoted as $Z_l'^i \in \mathbb{R}^{w\times h}$, where $I_l^i$ could be $R_l^i$ or $X_l^i$. Subsequently, a linear network followed by a Softmax function is used for the purpose of classification. The specific computational process is shown in (7). Where $Y'^i \in [0,1]^2$ indicates the confidence of real and fake, respectively, $P'^i \in [0,1]$ indicates the confidence of real, $W_l \in \mathbb{R}^{wh\times 2}$ and $b_l \in \mathbb{R}^2$ are the learnable parameters.

$$\begin{aligned} Z_l'^i &= \text{CNN}(I_l^i), \\ Y_l'^i &= \text{Softmax}(W_l Z_l'^i + b_l), \\ P_l'^i &= Y_{l,0}'^i. \end{aligned} \tag{7}$$

*Metric Discriminator:* The metrics discriminator, denoted as $D_m$, follows a structure similar to the Log Discriminator, with the distinction that its input is metric features $I_m^i$ that could be $R_m^i$ or $X_m^i$. The specific computational process is shown in (8). Where $W_m \in \mathbb{R}^{wh\times 2}$ and $b_m \in \mathbb{R}^2$ are the learnable parameters.

$$\begin{aligned} Z_m'^i &= \text{CNN}(I_m^i), \\ Y_m'^i &= \text{Softmax}(W_m Z_m'^i + b_m), \\ P_m'^i &= Y_{m,0}'^i. \end{aligned} \tag{8}$$

### E. Model Training

*Reconstruction Learning:* To effectively capture the distribution of normal data, we employ a reconstruction task as a fundamental component of our generator's training regimen. Specifically, for input data $X^i$, our generator is tasked with producing corresponding reconstructed data, as illustrated in (3) to (6). We refer to the difference between reconstructed values and real values as reconstructed errors. Then, we proceed to jointly train it using the reconstruction errors for both the log and metrics modalities, as shown in (9). Where $G_l()$ denotes the generator that outputs the logs portion, i.e., $R_l^i$, $G_m()$ denotes the the generator that outputs the metrics portion, i.e., $R_m^i$, MSE() is used to compute the Mean Squared Error (MSE) between two inputs, and $L_r$ represents the reconstruction loss, $N$ represents the total number of sliding windows.

$$L_r = \sum_i^N [\text{MSE}(G_l(X^i), X_l^i) + \text{MSE}(G_m(X^i), X_m^i)]. \tag{9}$$

*Adversarial Learning:* To enhance the fitting of hard samples without overfitting simple ones, we introduce the mechanism of adversarial learning. The discriminator is viewed as a dynamic detector for discerning hard and simple samples. When the generator performs poorly on hard samples, making it challenging to deceive the discriminator, the generator experiences higher training loss. Conversely, when the generator reconstructs sufficiently well on simple samples, easily deceiving the discriminator, the generator experiences lower training loss. Through the adversarial game between the generator and the discriminator, the model automatically adjusts the training loss concerning hard and simple samples. This automatic adjustment balances the progress of fitting hard and simple samples, leading to a better fit for the distribution of normal data.

The discriminator, which operates on both log and metrics modalities, is trained to differentiate between real data and generated data. We utilize cross-entropy loss for this purpose. The proposed adversarial learning architecture not only allows for the adjustment of fitting degrees between hard and simple multi-modal samples but also, with two separate discriminators, enables the adjustment of fitting degrees between the log and metric modality part of hard samples. This enhances the unified multi-modal representation learning. The adversarial loss for the discriminator $L_{ad}$ is defined as (10):

$$\begin{aligned} L_{ad} = -\sum_i^N [&log(D_l(X_l^i)) + log(D_m(X_m^i)) \\ &+ log(1 - D_l(G_l(X^i))) + log(1 - D_m(G_m(X^i)))]. \end{aligned} \tag{10}$$

The generator is trained to produce data that can effectively fool both $D_l$ and $D_m$. This is achieved by minimizing the following adversarial loss for the discriminator $L_{ag}$, as shown in (11):

$$\begin{aligned} L_{ag} = \sum_i^N &log(1 - D_l(G_l(X^i))) \\ &+ log(1 - D_m(G_m(X^i))). \end{aligned} \tag{11}$$

*Contrastive Learning:* Due to the complexity of multi-modal hard sample combinations, to further learn the distribution of normal log-metric multi-modal samples and enhance the distinguishing ability between normal and abnormal data, we employ unmatched data to simulate one of the abnormal scenarios in the

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

multi-modal setting. Matched data is used to simulate normal scenarios. Through contrastive learning, we aim to increase the reconstruction error of unmatched data and reduce the reconstruction error of matched data, thereby strengthening the distinction between normal and abnormal data.

*Construction of Unmatched Pairs:* To construct unmatched time blocks, we keep the log modality data fixed, search for metrics modality data that does not match, and generate unmatched time blocks. Specifically, for the $i_{th}$ time block $x^i = (x_l^i, x_m^i)$, we randomly select a time block $x^j = (x_l^j, x_m^j)$ from all time blocks. We require that $x_l^j \neq x_l^j$ and $abs(x_m^i, x_m^j) \geq \theta'$, and then generate a new unmatched time block $x'^i = (x_l^i, x_m^j)$, as shown in (12). Where $\theta'$ is a threshold to define the difference between $x_m^i$ and $x_m^j$. And then, for the $i_{th}$ matched slide window, we generate unmatched time blocks for each matched time block to achieve unmatched slide window $X'^i \in \mathbb{R}^{w\times(n+n')}$.

$$\begin{aligned} x'^i &= (x_l^i, x_m^j), \\ \exists\, x^j &= (x_l^j, x_m^j), \\ s.t.\ \ x_l^i &\neq x_l^j, \\ abs(x_m^i, x_m^j) &\geq \theta'. \end{aligned} \tag{12}$$

*Construction of Contrastive Loss:* We utilize a triple loss to construct the contrastive loss $L_c$, reducing the reconstruction loss for matched data while increasing the reconstruction loss for unmatched data, with the constraint that it does not exceed $\alpha$. The $\alpha$ represents the expected inter-class distance. This is expressed as shown in (13):

$$\begin{aligned} L_c = \sum_i^N \max(0, \mathrm{MSE}(G_m(X^i), X_m^i) \\ - \mathrm{MSE}(G_m(X'^i), X_m'^i) + \alpha). \end{aligned} \tag{13}$$

*Loss function:* In summary, during the training of the generator, we simultaneously employ reconstruction loss $L_r$, adversarial loss $L_{ag}$, and contrastive loss $L_c$ as indicated in (14). Here, we aim to minimize the whole loss of the generator $L_G$. Where $\lambda_1$ and $\lambda_2$ are weighting parameters that control the balance between the three losses.

$$L_G = L_r + \lambda_1 L_{ag} + \lambda_2 L_c. \tag{14}$$

During the training of the discriminator, we minimize the adversarial loss of discriminator $L_{ad}$ to distinguish whether data comes from real sources or the generator, as expressed in (15), where the $L_D$ indicates the training loss of the discriminator.

$$L_D = L_{ad}. \tag{15}$$

It is worth noting that during data preprocessing, it is crucial to determine the total number of log templates and monitoring metrics. When these quantities change, the proposed method requires retraining. Alternatively, if new log templates emerge but there is no desire to retrain the model, similarity matching can be employed to map the new log templates to known templates, ensuring the model's availability. In the future, there is a need to design anomaly detection methods capable of adapting to the continuous changes in the number of log templates and monitoring metrics.

TABLE I
THE OVERVIEW OF THE EVALUATION DATASETS

| Dataset | Log Message | Metric Length | Anomaly Ratio |
|---|---|---|---|
| Dataset A | 11141400 | 55707 | 26.00% |
| Dataset B | 5308847 | 42813 | 5.63% |
| Dataset C | 103645 | 8640 | 3.40% |

### F. Model Infering

When performing inference with our model, we utilize the generator to reconstruct input data and then determine anomalies based on the reconstruction error (anomaly score $Score$), as shown in (16), compared to a predefined threshold. The threshold is typically determined by binary searching for the best-f1 through whole dataset, like the prior researches [16], [17], [18], [20].

$$\begin{aligned} Score(X^i) = \mathrm{MSE}(X_l^i, (G_l(X^i))) \\ + \mathrm{MSE}(X_m^i, (G_m(X^i))). \end{aligned} \tag{16}$$

## IV. Evaluation

We evaluate UAC-AD by answering the following research questions (RQs):

- RQ1: How effective is UAC-AD in anomaly detection?
- RQ2: What is the contribution of each component of UAC-AD?
- RQ3: How does UAC-AD perform in terms of time and store efficiency?
- RQ4: How much is the method affected by parameter sensitivity?
- RQ5: Can we visually explain the impact of each component in our proposed model?

### A. Experiments Setup

*1) Datasets:* To evaluate the performance of UAC-AD, we conduct extensive experiments using two open-source data sets and a real industrial dataset from a large communication company, as shown in Table I. We partition 70% of each dataset for training and reserve the remaining 30% for testing.

- Dataset A is an in-lab dataset publicly released by [28] designed specifically for evaluating the effectiveness of multi-modal anomaly detection. It encompasses 11 metrics, sampled at 10-second intervals, forming time blocks of 10 seconds. Its log data includes 117 log templates.
- Dataset B is a complex multi-modal dataset simulated by an intelligent operations company. It serves as an evaluation benchmark for multi-modal anomaly detection under intricate scenarios. The dataset comprises metric and log data from four distinct services, with 85 metrics sampled every 30 seconds. The log data consists of 20 log templates. This dataset incorporates a variety of artificially injected

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

faults, such as CPU stress, memory stress, logging failure, and network failures.

- Dataset C is derived from a large-scale communication enterprise's real-time computing service platform with 21 online services. It is utilized to allocate computational resources for the company's online services. The dataset spans three days and includes 115 metrics, sampled every 30 seconds. It also features log data with 639 unique templates. Notably, this dataset includes 6 genuine anomalies induced by factors such as disk partition stress, CPU stress, and memory stress.

*2) Baseline:* To assess the effectiveness of our proposed method, we conducted comprehensive comparisons with various types of baseline methods. These comparisons encompassed different data sources, including single-modal log data, single-modal metric data, and multi-modal data. Additionally, we explored diverse training strategies, spanning supervised learning, semi-supervised learning, and unsupervised learning paradigms. Next, we will introduce them in detail.

- Deeplog [24] is a widely applied unsupervised log anomaly detection method. It uses the LSTM neural network to encode the log sequential information, predict the next log template, and determine the anomaly by judging whether the next log template is in top-k largest likely predicted log templates.
- LogRobust [25] is a supervised log anomaly detection method, which encodes the log sequence features, classifies them, and achieves robust and satisfactory performance on anomaly detection.
- OmniAnomaly [16] is an unsupervised anomaly detection based on metric. It seamlessly integrates Variational AutoEncoders (VAE) into time series anomaly detection. This integration serves to compress features effectively and learn the intricate distribution patterns of normal samples.
- MTAD-GAT [17] is an unsupervised anomaly detection that relies on metric data, which incorporates the graph attention neural network (GAT) to capture the spatial correlations among dimensions in multivariate time series.
- SCWarn [3] and DAM [33] are unsupervised multi-modal anomaly detection methods that adopt the LSTM to encode the log and metric sequential features and fuse the features by concatenation or cross attention operation.
- Hades [28] is a semi-supervised multi-modal anomaly detection approach. This method employs FastText [43] for log text embedding, utilizes Transformer for log feature extraction, employs CNNs for metric feature extraction, and integrates these features through concatenation operations.
- Union: To further investigate the effect of our multi-modal method, we also design the ensemble methods as baselines. We split our model into two single models: the log model and the metric model. Then, we use them for anomaly detection, respectively, and union their results as the final results: If one of the log model or the metric model detected an anomaly for a time block, this anomaly is considered to have been detected.
- Intersection: Similar to the *Union* baseline, the difference is we take the intersection of the results from the log model and the metric model as the final results: Both the log model and metric model detected an anomaly for a time block, this anomaly is considered to have been detected.

*3) Evaluation Measurements:* We apply the widely used measurements to assess our method. We adopt true positive (TP), false positive(FP), and false negative(FN) to label the anomaly detection results according to the ground truth. *TP* denotes a failure both confirmed by ground truth and detected by a method. *FP* is a normal time block that is falsely identified by a method. *FN* is a missed anomaly that should have been detected. We calculate $precision = TP/(TP + FP)$, $recall = TP/(TP + FN)$, and $F1 - score = (2 \cdot precision \cdot recall)/(precision + recall)$.

*4) Implementation:* UAC-AD is implemented in Pytorch, and all of the experiments are conducted on a Linux Server with an NVIDIA GeForce GTX 3090 GPU via Python 3.8. As for the hyper-parameters, UAC-AD adopts the unmatched $\alpha = 0.16$, the window size $w = 40$, and the hidden size $h = 32$, and these hyper-parameters are described in detail in section IV-E. We use the Adam optimizer [44] with an initial learning rate of 0.001. We set the $\lambda_1 = \lambda_2 = 1$ to treat different learning strategies equally. We set the $\theta' = 0.15$ to balance the distinguishment between matched and unmatched pair samples and the time-coast for finding the unmatched pair samples following (12). The batch size is 256, and the epoch of each training phase is dynamically adjusted by an early stop strategy.

For hyper-parameters of baselines, we adopt the public implementations of [3], [16], [17], [24], [25], [28], and determine the hyper-parameter combination achieving the highest test *F1-score* for each baseline.

### B. RQ1: The Overall Performance of the Proposed Method

To assess the effectiveness of our proposed method, we compared it with various types of baseline methods, including single-modal methods for logs and metrics, multi-modal methods, and ensemble learning methods, as shown in Table II. The results demonstrate that our proposed approach outperforms the baseline methods on Dataset B and Dataset C, achieving improvements of 4.5 and 19.3 percentage points, respectively. On Dataset A, our method surpasses existing unsupervised state-of-the-art methods by 6.7 points and approaches the performance of existing multi-modal semi-supervised state-of-the-art methods. This highlights the effectiveness of our proposed approach.

Compared to single-modal methods that consider only log or metric data, our method simultaneously incorporates information from multiple modalities, leading to significant performance gains. In contrast to existing multi-modal unsupervised methods, our approach enhances fitness to hard samples and further learns the distribution of normal multi-modal data through adversarial contrastive learning. Consequently, it improves the discriminative power between normal and anomalous data, resulting in additional performance gains. It is interesting to note that the proposed unsupervised method generally outperforms the semi-supervised method Hades. This may be because semi-supervised methods like Hades are prone to be influenced by sparsely labeled data, making it difficult to adapt to unseen anomalies. In contrast, our proposed unsupervised method learns implicit patterns of multi-modal data more comprehensively through a

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE OVERALL PERFORMANCE COMPARISON OF OUR METHOD UAC-AD AND SEVERAL STATE-OF-THE-ART METHODS

| Methods | Source | Training strategy | Dataset A | | | Dataset B | | | Dataset C | | | Avg |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | | F1 | Rec | Pre | F1 | Rec | Pre | F1 | Rec | Pre | F1 |
| DeepLog | log | unsupervised | 0.259 | 0.386 | 0.194 | 0.547 | 0.415 | 0.802 | 0.085 | 0.854 | 0.045 | 0.297 |
| LogRobust | log | supervised | 0.841 | 0.847 | 0.835 | 0.779 | 0.838 | 0.727 | 0.501 | 0.333 | 0.999 | 0.707 |
| OmniAnomaly | metric | unsupervised | 0.681 | 0.788 | 0.601 | 0.288 | 0.457 | 0.211 | 0.494 | 0.328 | 0.999 | 0.487 |
| MTAD-GAT | metric | unsupervised | 0.719 | 0.764 | 0.678 | 0.384 | 0.466 | 0.326 | 0.512 | 0.344 | 0.999 | 0.538 |
| SCWarn | multi-modal | unsupervised | 0.812 | 0.882 | 0.752 | 0.393 | 0.531 | 0.311 | 0.440 | 0.344 | 0.611 | 0.548 |
| DAM | multi-modal | unsupervised | 0.839 | 0.852 | 0.827 | 0.554 | 0.844 | 0.450 | 0.747 | 0.672 | 0.840 | 0.713 |
| Hades | multi-modal | semi-supervised | **0.918** | 0.967 | 0.875 | 0.811 | 0.832 | 0.792 | 0.741 | 0.667 | 0.833 | 0.823 |
| Union | multi-modal | unsupervised | 0.825 | 0.992 | 0.706 | 0.913 | 0.972 | 0.861 | 0.631 | 0.999 | 0.460 | 0.789 |
| Intersection | multi-modal | unsupervised | 0.869 | 0.826 | 0.917 | 0.438 | 0.283 | 0.963 | 0.512 | 0.344 | 0.999 | 0.606 |
| **UAC-AD** $(Our)$ | multi-modal | unsupervised | 0.879 | 0.857 | 0.902 | **0.958** | 0.959 | 0.957 | **0.934** | 0.999 | 0.877 | **0.923** |

*F1*, *Rec*, and *Pre* represent the F1 score, Recall, and Precision evaluation metrics, respectively. Higher values indicate better performance. The best and second-best results are highlighted with bold font and underlining for clarity.
Unlike the results in Hades, we adopted the widely-used adjustment strategy [16],[17],[18],[20] for all methods: If a certain time point in a continuous abnormal segment is detected, all anomalies in that abnormal segment are considered to have been correctly detected. The Adjustment strategy aligns with the practical application scenarios. In real-world applications, once an anomaly is detected at a certain moment, it is sufficient to identify the anomaly without necessarily detecting the entire anomalous period.

TABLE III
THE ABLATION STUDY ON EACH COMPONENT OF UAC-AD OVER THREE DATASETS

| Method | Source | | Strategies | | Dataset A | | | Dataset B | | | Dataset C | | | Avg |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | *Log* | *Metric* | *Adversarial* | *Contrastive* | F1 | Rec | Pre | F1 | Rec | Pre | F1 | Rec | Pre | F1 |
| Method1 | ✓ | | | | 0.791 | 0.788 | 0.797 | 0.875 | 0.792 | 0.977 | 0.741 | 0.999 | 0.587 | 0.802 |
| Method2 | | ✓ | | | 0.856 | 0.941 | 0.786 | 0.479 | 0.396 | 0.643 | 0.789 | 0.672 | 0.956 | 0.708 |
| Method3 | ✓ | | ✓ | | 0.843 | 0.799 | 0.892 | 0.873 | 0.787 | 0.979 | 0.795 | 0.999 | 0.666 | 0.837 |
| Method4 | | ✓ | ✓ | | 0.869 | 0.857 | 0.882 | 0.485 | 0.406 | 0.639 | 0.804 | 0.672 | 0.999 | 0.719 |
| Method5 | ✓ | ✓ | | | 0.864 | 0.847 | 0.883 | 0.887 | 0.915 | 0.879 | 0.454 | 0.344 | 0.667 | 0.735 |
| Method6 | ✓ | ✓ | ✓ | | 0.876 | 0.944 | 0.819 | 0.941 | 0.934 | 0.946 | 0.811 | 0.999 | 0.681 | 0.876 |
| Method7 | ✓ | ✓ | | ✓ | 0.856 | 0.861 | 0.851 | 0.927 | 0.914 | 0.945 | 0.865 | 0.999 | 0.762 | 0.883 |
| Method8 $(Our)$ | ✓ | ✓ | ✓ | ✓ | **0.879** | 0.857 | 0.902 | **0.958** | 0.959 | 0.957 | **0.934** | 0.999 | 0.877 | **0.923** |

*Log* and *Metric* represent the different data sources for the model. *Adversarial* and *Contrastive* correspond to the Adversarial learning and Contrastive learning strategies, as introduced in subsection III-E. Reconstruction learning is the default learning strategy.

reconstruction task, without being affected by the scarcity of labeled data. This leads to superior performance compared to Hades. Moreover, compared to methods directly performing result fusion, our proposed approach utilizes deeper feature fusion, leading to more substantial improvements in performance. In terms of the recall metric, only the union method surpasses our method. This is because the union method splits our method into two models, one for logs and one for metrics, and combines the results using a logical OR operation. Consequently, it is less likely to miss anomalies, resulting in a particularly high recall rate. However, it may also lead to more false positives since anomalies in either logs or metrics can trigger an alert, even if the system is not actually experiencing an anomaly. Similarly, only the intersection method outperforms our method in precision. This is because the intersection method only considers cases where both models detect anomalies as actual anomalies. As a result, its precision is high, but it tends to miss many anomalies, resulting in a lower recall rate.

### C. RQ2: The Ablation Study of the Proposed Method

To assess the effectiveness of each component in our proposed method, we conducted comparisons using different variants of our approach, as shown in Tables III and IV.

TABLE IV
THE COMPARATIVE ANALYSIS OF DIFFERENT FEATURE FUSION STRATEGIES SHOWCASING THE F1 SCORE EVALUATION METRIC

| Fusion | Dataset A | Dataset B | Dataset C | Average |
|---|---|---|---|---|
| CA | 0.848 | 0.915 | 0.811 | 0.858 |
| SA | 0.854 | 0.905 | 0.895 | 0.884 |
| CO | 0.839 | 0.942 | 0.859 | 0.881 |
| MSA $(Our)$ | **0.879** | **0.958** | **0.934** | **0.923** |

*CA*, *SA*, *CO*, and *MSA* represent the utilization of cross-attention, self-attention, concatenation, and multi-modal self-attention, respectively, to merge multi-modal features.

*Effectiveness of Adversarial Learning:* Table III demonstrates that introducing adversarial learning significantly enhances the F1 score of anomaly detection, both in single-modal (comparing Method 1 with Method 3, or Method 2 with Method 4) and multi-modal scenarios (comparing Method 5 with Method 6). This improvement is attributed to adversarial learning's ability to help the generator balance the training of hard samples and simple samples, improving the fitting of the distribution of normal data, and thereby enhancing anomaly detection effectiveness. After the introduction of GAN, the significant increase in F1 score in the multi-modal scenario is notably higher than the improvements seen in each individual modality. This is attributed

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

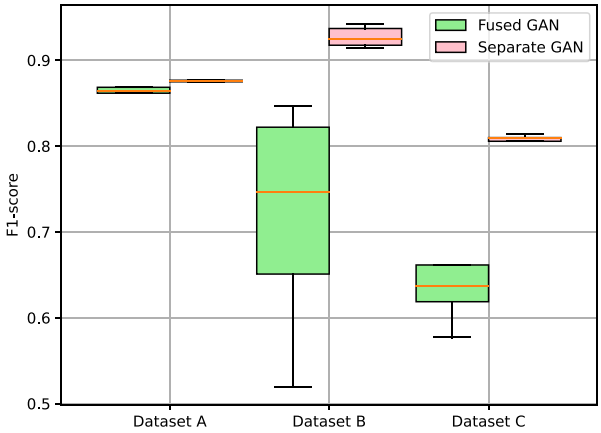



Fig. 4. The comparison between Fused GAN and Separate GAN (i.e., Method6 in Table III) with 10 times running. The discriminator of Fused GAN utilizes two 1D CNNs to embed data from the log and metric modalities separately. These embeddings are then fused using MSA to obtain a unified representation, which is finally used by the classification layer to discriminate whether the input data is real or reconstructed.

to the fact that separate discriminators also regulate the fitting degrees of different modality parts of hard samples, facilitating the model to better learn the normal patterns of log and metric modal data. Fig. 4 provides a more straightforward comparison, showing that compared to Fused GAN, Separate GAN exhibits better anomaly detection performance across all three datasets, validating the effectiveness of fine-grained adversarial learning.

*Effectiveness of Contrastive Learning:* Comparing Method 5 with Method 7 in Table III, the results indicate that introducing contrastive learning significantly improves the F1 score of anomaly detection. Contrastive learning achieves this through the matched pairs and the unmatched pairs, enhancing the learning of the distribution of multi-modal normal data. This increases the discriminative power between normal and anomalous data, improving anomaly detection effectiveness. Additionally, comparing Methods 5, 6, 7, and 8, the results show that the combination of contrastive learning and adversarial learning leads to further performance improvements. This demonstrates the complementary contributions of these two components, confirming their effectiveness.

*Effectiveness of MSA Design:* To evaluate the effectiveness of the Multi-Modal Self-Attention (MSA) design, we compared it with common multi-modal feature fusion methods, as shown in Table IV. The results highlight the significant effectiveness of MSA compared to other feature fusion mechanisms. This superiority is due to MSA's ability to capture not only the correlations between latent variables across different modalities but also the correlations between latent variables within single modalities. Consequently, MSA achieves superior feature fusion results.

### D. RQ3: The Time and Storage Efficiency of the Proposed Method

To evaluate the time and storage efficiency of our proposed method, we compared the inference time and the number of parameters required with various baseline methods, as shown in Table V. *Time Efficiency:* The results demonstrate that although the proposed method may not excel in training time due to its focus on acquiring better representations of multimodal data, it still demonstrates efficient model inference time, making it well-suited for real-time applications. *Store Efficiency:* The results show that our proposed method requires fewer parameters than most baseline models (LogRobust, OmniAnomaly, MTAD-GAT, SCWarn, and Hades), indicating its relatively small storage overhead and facilitating the deployment of the model. Although DeepLog has a smaller number of parameters, the substantial performance gains of our proposed method make it quite suitable for practical applications in real-world scenarios.

TABLE V
THE COMPARISON OF THE TIME AND STORAGE EFFICIENCY BETWEEN UAC-AD AND BASELINE METHODS

| | *Params*(M) | *Training / Inference Time*(s) | | |
|---|---|---|---|---|
| | | Dataset A | Dataset B | Dataset C |
| DeepLog | **0.06** | 34.12 / 59.40 | 89.72 / 322.10 | 2.31 / 0.68 |
| LogRobust | 0.27 | 3.21 / 1.20 | **3.22** / 0.80 | **0.33** / 0.14 |
| OmniAnomaly | 2.63 | 78.31 / 144.40 | 282.91 / 101.10 | 28.72 / 18.90 |
| MTAD-GAT | 0.38 | 2.82 / 1.40 | 21.70 / 3.77 | 1.92 / 0.24 |
| SCWarn | 0.12 | **1.61** / 10.40 | 8.32 / 9.60 | 1.11 / 1.60 |
| Hades | 1.51 | 23.71 / 3.00 | 33.61 / 1.92 | 1.41 / 0.23 |
| *Our* | 0.09 | 52.71 / **0.28** | 61.52 / **0.22** | 1.13 / **0.03** |

*Params* represents the metric for the number of model parameters required, and *Training (Inference) Time* indicates the metric for the time taken to train (infer) the entire training (test) set, respectively. Smaller values for both metrics are indicative of superior performance.

### E. RQ4: The Parameter Sensitivity of the Proposed Method

To evaluate the sensitivity of our method's hyperparameters, we calculated the average F1-score under different hyperparameter settings, as shown in Fig. 5.

*The inter-class distances in contrastive learning:* The parameter $\alpha$ introduced in (13) holds significant importance as a hyperparameter in our contrastive learning module, exerting a substantial impact on the performance of *UAC-AD*. We systematically varied $\alpha$ across a range from 0.02 to 0.2. Interestingly, as $\alpha$ increased, there was a noticeable improvement in F1-score for Dataset C. However, this improvement was limited in Dataset A. This divergence in performance may be attributed to the diverse nature of matched pairs within different datasets. In the case of Dataset A, the unmatched pairs may be similar to the matched pairs. Consequently, enforcing a clear distinction between positive and negative samples could potentially compromise the performance of *UAC-AD*. To strike a balance in performance across all datasets, we opted for $\alpha = 0.16$, a choice that resulted in a harmonious and optimal performance.

*The time window sizes of adversarial learning generator:* We systematically varied $w$ within the range of 10 to 100. As observed, increasing the value of $w$ initially led to an improvement in the F1-score, followed by a subsequent decline. This behavior can be attributed to the fact that a larger time window can provide a richer contextual understanding, thereby enhancing the model's performance. However, if the time window becomes excessively large, the model encounters challenges in capturing

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

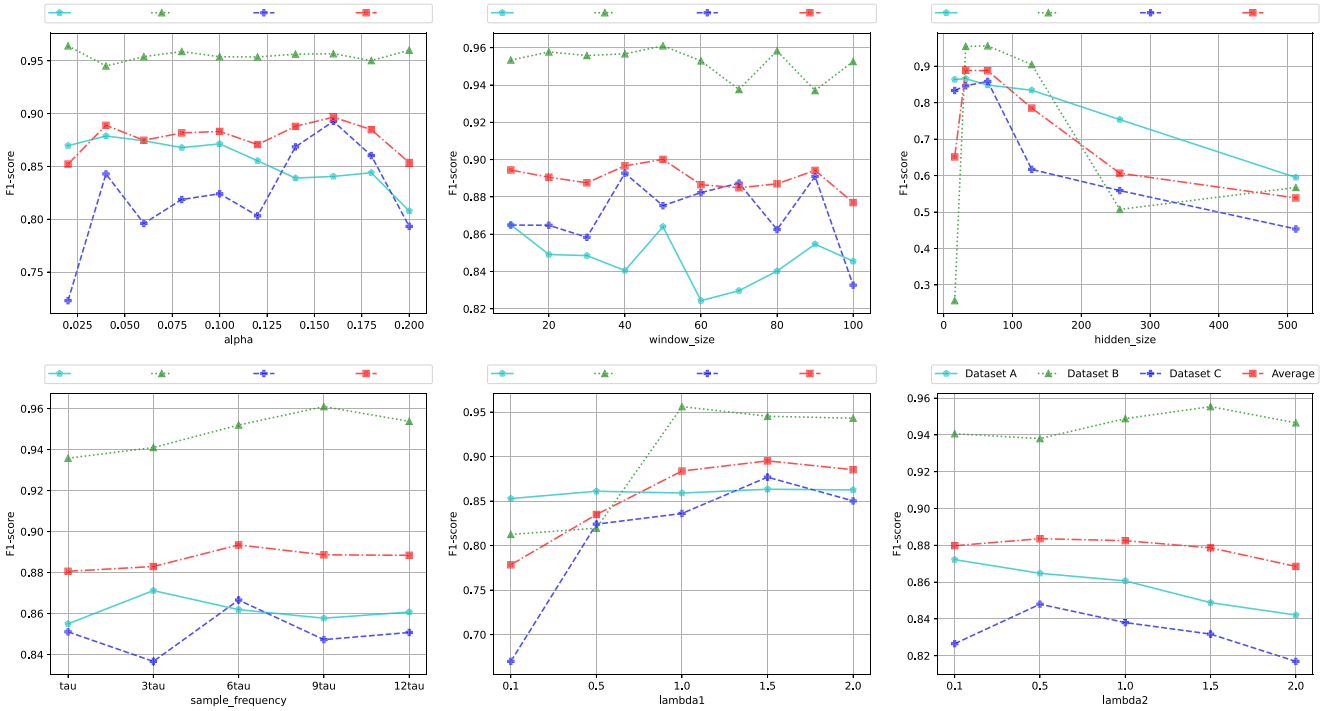



Fig. 5. The evaluation of the impact of different inter-class distances $\alpha$ in contrastive learning, various time window sizes $w$, different hidden layer sizes $h$ of adversarial learning generators, different sample frequency, different $\lambda_1$ and $\lambda_2$ on anomaly detection performance across three datasets.

intricate and extensive temporal correlations, resulting in a decrease in overall performance. To attain optimal model performance, we carefully evaluated the impact of different window sizes and determined that $w = 50$ yielded the best results.

*The hidden layer sizes of adversarial learning generator:* We systematically varied $h$ within the range of $[16, 32, 64, 128, 256, 512]$. Our observations revealed that increasing the value of $h$ initially resulted in an improvement in the F1-score, followed by a rapid decline. This pattern can be attributed to the fact that larger hidden layer representation vectors can capture more intricate features, leading to enhanced model performance. However, an excessively large hidden layer representation might encode excessive detail in the features, resulting in a lack of high-level feature representation and a subsequent decline in overall model performance. Our evaluation indicated that $h = 32$ yielded the most favorable results, striking a balance between capturing detailed features and maintaining high-level feature representation in the model.

*The weights of the training loss:* We systematically varied $\lambda_1$ and $\lambda_2$ within the range of $[0.1, 0.5, 1.0, 1.5, 2]$. The results indicate that as $\lambda_1$ increases, the overall F1 score of the model initially increases, stabilizing around $\lambda_1 = 1.0$. This suggests the effectiveness of adversarial learning in anomaly detection. Therefore, we select $\lambda_1 = 1.0$ to achieve better results. Meanwhile, as $\lambda_2$ increases, the overall F1 score of the model transitions from stability to decline. This could be due to the unmatched pairs still retaining certain temporal characteristics of metrics, and excessively magnifying the reconstruction performance of unmatched pairs may disrupt the learning of underlying multi-modal patterns. Moreover, as $\lambda_2$ increases, there are variations in model performance across different datasets. Dataset B and dataset C initially exhibit an increase followed by a decrease in F1 score, demonstrating the effectiveness of introducing contrastive learning. However, dataset A consistently shows a decline with increasing $\lambda_2$. The underlying reason for this trend may still be that the unmatched pairs are similar to the matched pairs. Therefore, we choose $\lambda_2 = 1.0$ to achieve better results.

*Sampling frequency:* Although the original sampling frequencies of different datasets have been fixed, for example, dataset A is sampled every 10 seconds, while datasets B and C are sampled every 30 seconds, we attempted to explore the impact of different sampling frequencies on the proposed method. Let $\tau$ represent the original sampling interval of the dataset. We systematically varied the sampling intervals as $[\tau, 3\tau, 6\tau, 9\tau, 12\tau]$. The results show that as the sampling interval increases, the overall F1 score of the proposed method tends to stabilize, indicating that the proposed method is insensitive to sampling frequency.

### F. RQ5: The Interpretability of the Proposed Method

In order to better assess the effectiveness of the components we designed, we visually demonstrated the impact of different components on the reconstruction errors of input data, as shown in Fig. 6. In this context, normal samples interspersed with anomaly samples can be regarded as hard samples. Introducing the *Adversarial Learning* mechanism significantly reduced the reconstruction errors of normal data. This indicates that our designed *Adversarial Learning* enhances the model's balance between hard samples and simple samples, and strengthens the learning of normal data distribution. With the introduction of

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

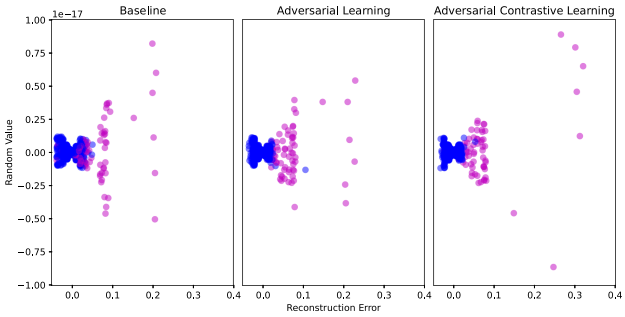



Fig. 6. The visualization of reconstruction errors for test set samples obtained using various learning strategies on Dataset B. The magenta dots indicate anomaly samples, while the blue dots represent normal samples. The horizontal axis signifies the reconstruction error, while the vertical axis is associated with random values to enable a two-dimensional visualization of the reconstruction error.

the *Contrastive Learning* mechanism, the reconstruction errors of abnormal data significantly increased, enhancing the distance between normal and abnormal samples. This demonstrates that our designed *Contrastive Learning* improves the model's understanding of multi-modal normal data. By expanding the distance between positive and negative samples, it enhances the distinguishability between normal and abnormal samples.

## V. Discussion

### A. Lessons Learned

*Complementarity of Multi-Source Data for Anomaly Detection in Microservices Systems:* While impressive results can be achieved with single-modal data, as demonstrated by the metric-based anomaly detection (e.g., Method1) in Dataset A, or the log-based anomaly detection (e.g., Method2) in Dataset B, as shown in Table III, the effective fusion and learning of multi-modal data still lead to a notable improvement in performance. This emphasizes the complementary nature of multi-modal data in microservices systems for anomaly detection. This insight suggests that future research in microservices system anomaly detection should focus more on the detection of anomalies using multi-modal data rather than relying solely on single-modal data.

*Utility of Middleware Service Logs for Anomaly Detection in Other Services:* In Dataset C, we collected metrics and logs from a total of 21 services, with most working services not generating logs. The majority of collected logs were from a middleware service of a message processor. Despite faults potentially occurring in working services that do not generate logs (for instance, if a container of a working service crashes and restarts, and coincidentally not reflected in the metric data due to inherent sampling intervals and imperfect configuration [45]), anomalies can still be detected using logs from the message processor, as evident from the log-based anomaly detection (Method1 and Method3) recall rate of nearly 100% in Table III. This indicates that logs from middleware services can be helpful for detecting anomalies in other services. This insight suggests that future microservices' multi-modal anomaly detection should not only consider metrics and logs from the target service but should also take into account logs from middleware services to aid in the detection process.

### B. The Generality of the Proposed Method

The proposed method exhibits a certain degree of generalization. First, in terms of feature quantity, our method collects all data from different datasets and inputs them into the generator, which automatically extracts a fixed number of features (determined by the hyperparameter hidden size). Second, in terms of anomaly types, our method does not restrict the types of anomalies it can detect and can identify anomalies in different microservices systems. Therefore, our method demonstrates a certain level of generalization.

### C. Threats to Validity

Threats to internal validity come from the annotation of anomalies. Labeling principles vary depending on the purpose, system, person, and so on. Thus, the labeling may exist as inconsistency when it comes to non-extreme anomalies or rare patterns, such as the slightly steep ups and downs in metrics, metric variations between normal and abnormal fluctuation, log statements rarely occurring, etc. To alleviate these concerns, the adopted datasets are verified by experienced annotators, and we further evaluate the correctness of the labeling of each dataset.

The external threat mainly comes from our datasets. Though our experiments are evaluated on three datasets, it is yet unknown whether the effectiveness of our method UAC-AD can be generalized across all other datasets. To mitigate the threat, we use different datasets with representative workloads and typical faults to evaluate UAC-AD, and the experimental results show the UAC-AD can work not only on simulated data but also on real industrial data. We will also evaluate our method on more datasets in the future.

## VI. Conclusion

In this paper, we primarily address the hard sample problem in anomaly detection of log-metric modalities in microservices systems. To tackle the complexity of their combinations, we design contrastive learning to capture the patterns of log-metric combinations during normal system operation, while distancing from abnormal combinations. To address the issue of inconsistent convergence speeds, we employ an adversarial learning framework to adjust the learning speeds of hard and simple samples automatically. Moreover, we use separate discriminators to regulate the fitting degree of each modality part of the multi-modal hard samples. Extensive experiments are conducted to validate the efficacy of our proposed method.

## References

[1] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, "Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2023, pp. 553–565.

[2] C. Lee, T. Yang, Z. Chen, Y. Su, and M. R. Lyu, "Eadro: An end-to-end troubleshooting framework for microservices on multi-source data," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.*, 2023, pp. 1750–1762.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.

[3] N. Zhao et al., "Identifying bad software changes via multimodal anomaly detection for online service systems," in *Proc. 29th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2021, pp. 527–539.
[4] S. Jin et al., "Unsupervised hard example mining from videos for improved object detection," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 307–324.
[5] C. Zhang, Y. Wang, and W. Tan, "MTHM: Self-supervised multi-task anomaly detection with hard example mining," *IEEE Trans. Instrum. Meas.*, vol. 72, pp. 1–13, 2023.
[6] S. Kong, J. Ai, and M. Lu, "CL-MMAD: A contrastive learning based multimodal software runtime anomaly detection method," *Appl. Sci.*, vol. 13, no. 6, 2023, Art. no. 3596.
[7] "UAC-AD," [Online]. Available: https://github.com/lhysgithub/UAC-AD
[8] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2000, pp. 93–104.
[9] J. Tang, Z. Chen, A. -C. Fu, and D. W. Cheung, "Enhancing effectiveness of outlier detections for low density patterns," in *Proc. Adv. Knowl. Discov. Data Mining: 6th Pacific-Asia Conf.*, PAKDD 2002 Taipei, Taiwan, Springer, 2002, pp. 535–548.
[10] B. Zong et al., "Deep autoencoding gaussian mixture model for unsupervised anomaly detection," in *Proc. Int. Conf. Learn. Representations*, 2018.
[11] T. Yairi, N. Takeishi, T. Oda, Y. Nakajima, N. Nishimura, and N. Takata, "A data-driven health monitoring method for satellite housekeeping data based on probabilistic clustering and dimensionality reduction," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 53, no. 3, pp. 1384–1401, Jun. 2017.
[12] D. M. Tax and R. P. Duin, "Support vector data description," *Mach. Learn.*, vol. 54, pp. 45–66, 2004.
[13] L. Ruff et al., "Deep one-class classification," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4393–4402.
[14] L. Shen, Z. Li, and J. Kwok, "Timeseries anomaly detection using temporal hierarchical one-class network," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 13016–13026.
[15] D. Park, Y. Hoshi, and C. C. Kemp, "A multimodal anomaly detector for robot-assisted feeding using an lstm-based variational autoencoder," *IEEE Robot. Automat. Lett.*, vol. 3, no. 3, pp. 1544–1551, Jul. 2018.
[16] Y. Su, Y. Zhao, C. Niu, R. Liu, W. Sun, and D. Pei, "Robust anomaly detection for multivariate time series through stochastic recurrent neural network," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 2828–2837.
[17] H. Zhao et al., "Multivariate time-series anomaly detection via graph attention network," in *Proc. IEEE Int. Conf. Data Mining*, 2020, pp. 841–850.
[18] J. Zhan et al., "Stgat-mad: Spatial-temporal graph attention network for multivariate time series anomaly detection," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process.*, 2022, pp. 3568–3572.
[19] D. Li, D. Chen, B. Jin, L. Shi, J. Goh, and S.-K. Ng, "MAD-GAN: Multivariate anomaly detection for time series data with generative adversarial networks," in *Proc. Int. Conf. Artif. Neural Netw.*, Springer, 2019, pp. 703–716.
[20] J. Audibert, P. Michiardi, F. Guyard, S. Marti, and M. A. Zuluaga, "USAD: Unsupervised anomaly detection on multivariate time series," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2020, pp. 3395–3404.
[21] W. Xu, "System problem detection by mining console logs," University of California, Berkeley, Tech. Rep. UCB/EECS-2010–112, 2010.
[22] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *Proc. 38th Int. Conf. Softw. Eng. Companion*, 2016, pp. 102–111.
[23] S. He, Q. Lin, J.-G. Lou, H. Zhang, M. R. Lyu, and D. Zhang, "Identifying impactful service system problems via log analysis," in *Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2018, pp. 60–70.
[24] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1285–1298.
[25] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2019, pp. 807–817.
[26] W. Guan, J. Cao, Y. Gu, and S. Qian, "GRASPED: A GRU-AE network based multi-perspective business process anomaly detection model," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3412–3424, Sep./Oct. 2023.
[27] Y. Fu, K. Liang, and J. Xu, "MLog: Mogrifier LSTM-based log anomaly detection approach using semantic representation," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3537–3549, Sep./Oct. 2023.
[28] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, and M. R. Lyu, "Heterogeneous anomaly detection for software systems via semi-supervised cross-modal attention," 2023, *arXiv:2302.06914*.
[29] C. Zhang et al., "DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning," in *Proc. 44th Int. Conf. Softw. Eng.*, 2022, pp. 623–634.
[30] J. Bogatinovski and S. Nedelkoski, "Multi-source anomaly detection in distributed it systems," in *Proc. Int. Conf. Serv.-Oriented Comput.*, Springer, 2020, pp. 201–213.
[31] J. Huang, Y. Yang, H. Yu, J. Li, and X. Zheng, "Twin graph-based anomaly detection via attentive multi-modal learning for microservice system," in *Proc. 38th IEEE/ACM Int. Conf. Autom. Softw. Eng.*, 2023, pp. 66–78.
[32] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "Swisslog: Robust anomaly detection and localization for interleaved unstructured logs," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 4, pp. 2762–2780, Aug. 2023.
[33] Y. Chen, M. Yan, D. Yang, X. Zhang, and Z. Wang, "Deep attentive anomaly detection for microservice systems with multimodal time-series data," in *Proc. IEEE Int. Conf. Web Serv.*, 2022, pp. 373–378.
[34] Q. Zhang, J. Han, L. Cheng, B. Zhang, and Z. Gong, "Approach to anomaly detection in microservice system with multi-source data streams," *ZTE Commun.*, vol. 20, no. 3, pp. 85–92, 2022.
[35] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 2672–2680.
[36] S. Akcay, A. Atapour-Abarghouei, and T. P. Breckon, "GANomaly: Semi-supervised anomaly detection via adversarial training," in *Proc. 14th Asian Conf. Comput. Vis.*, Perth, Australia, Springer, 2019, pp. 622–637.
[37] T. Schlegl, P. Seeböck, S. M. Waldstein, U. Schmidt-Erfurth, and G. Langs, "Unsupervised anomaly detection with generative adversarial networks to guide marker discovery," in *Proc. Int. Conf. Inf. Process. Med. Imag.*, Springer, 2017, pp. 146–157.
[38] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2015, pp. 815–823.
[39] S.-J. Peng, Y. Fan, Y.-M. Cheung, X. Liu, Z. Cui, and T. Li, "Towards efficient cross-modal anomaly detection using triple-adaptive network and bi-quintuple contrastive learning," *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 8, no. 1, pp. 697–709, Feb. 2024.
[40] P. He, J. Zhu, Z. Zheng, and M. R. Lyu, "Drain: An online log parsing approach with fixed depth tree," in *Proc. IEEE Int. Conf. web Serv.*, 2017, pp. 33–40.
[41] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
[42] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016, *arXiv:1607.06450*.
[43] A. Joulin et al., "Fasttext.zip: Compressing text classification models," 2016, *arXiv:1612.03651*.
[44] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2014, *arXiv:1412.6980*.
[45] M. Ma et al., "An empirical investigation of missing data handling in cloud node failure prediction," in *Proc. 30th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, 2022, pp. 1453–1464.



**Hongyi Liu** received the MS degree from Peking University, in 2021. Currently, he is working toward the PhD degree with the School of Software & Microelectronics, Peking University. His research interests include could computing, software reliability, anomaly detection, and AI for software engineering.



**Xiaosong Huang** received the BS degree from the Beijing Institute of Technology, in 2021. He is currently working toward the PhD degree with the School of Computer Science at Peking University. His research interests include AIOps, software reliability, AI for software engineering.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.



**Mengxi Jia** received the BS degree from Central South University, China. She is currently working toward the PhD degree with the School of Software and Microelectronic, Peking University, Beijing, China. Her research interests include machine learning, deep representation learning, AI for software engineering.



**Tong Jia** received the PhD degree in software engineering from Peking University, China, in 2019. He is currently an assistant research professor with the institute for artificial intelligence, Peking University, China. His research interests include AIOps, anomaly detection, etc. He published more than 30 papers in international journals and conferences.



**Jing Han** received the MS degree from the Nanjing University of Aeronautics and Astronautics, in 2000. Currently, she is working with ZTE. She is responsible for AIOPS in the communications and cloud domains.



**Zhonghai Wu** received the PhD degree from Zhejiang University, Hangzhou, China, in 1997. Previously he worked as a postdoctoral and associate professor with the Institute of Computer Science and Technology, Peking University, Beijing, China. He is currently a professor and dean with the School of Software and Microelectronics, Peking University. His research interests include cloud computing, Big Data, AI and information security.



**Ying Li** (Member, IEEE) received the PhD degree in computer science and engineering from Northwestern Polytechnical University (NWPU), China, in 2001. She is currently a professor with the National Engineering Research Center of Software Engineering and the School of Software and Microelectronics, Peking University, China. Before joining PKU in 2012, she worked as a STSM and senior manager leading the department of distributed computing in IBM China Research Center. Her research interests include distributed computing and dependability engineering. She filed 40+ US/CN granted patents and was awarded as "IBM Master Inventor". She published more than 90 academic papers in international journals and conferences and served as PC member of several international conferences and reviewer of international journals.

Authorized licensed use limited to: Universidad Catolica Santa Maria. Downloaded on November 08,2025 at 22:36:17 UTC from IEEE Xplore. Restrictions apply.